$mod ::= \textbf{MODULE}(mname, dep\ ..., defn\ ...)$
$M ::= mod\ ...$
$mname ::= name$
$dep ::= mname@ph$
$deps ::= dep\ ...$
$defn ::= \langle dname@ph, kind, ast\rangle$
$defns ::= defn\ ...$
$dname ::= name$
$kind ::= \textbf{VALUE} \mid \textbf{MACRO}$
$ph ::= number$

$binding ::= \langle ph, mname, dname\rangle$
$bindings ::= binding\ ...$

$\Sigma ::= \{cell\ ...\}$
$cell ::= \langle binding, kind, val\rangle$
$\quad \mid \langle mname@ph\rangle$

$s\text{-}exp ::= name \mid (\ s\text{-}exp\ ...\ )$
$name ::=$ a token such as `x`, `egg`, or `lambda`

$ast ::= val$
$\quad \mid \textbf{REF}(binding)$
$\quad \mid var$
$\quad \mid \textbf{APP}(ast, ast, ...)$
$val ::= func$
$\quad \mid \textbf{QUOTE}(name)$
$\quad \mid \textbf{SYNTAX}(stx)$
$\quad \mid \textbf{UNBOUND}$
$func ::= \textbf{LAMBDA}(var\ ..., ast)$
$var ::= \textbf{VAR}(name)$

$stx ::= id \mid (\ stx\ ...\ )$
$id ::= \textbf{STX}(name, bindings)$

$body ::= stx@ph$

---

cmodule : $s\text{-}exp\ M \to M$

cmodule⟦ (`module` $mname\ mname_{init}\ s\text{-}exp\ ...$) , M⟧
= cbody⟦$body\ ..., mname,\ mname_{init}@0, \varepsilon, M, \Sigma_{init}$⟧
where Σ_{init} = visit⟦$mname_{init}, 0, M, \varnothing$⟧,
$bindings$ = import⟦$mname_{init}, 0, M$⟧,
$body\ ...$ = wrap⟦$s\text{-}exp, bindings$⟧@0 ...

---

cbody : $body\ ...\ mname\ dep\ ...\ defn\ ...\ M\ \Sigma \to M$

cbody⟦$\varepsilon, mname, deps, defns, M, \Sigma$⟧ = $M\left[mname \leftarrow \textbf{MODULE}(mname, deps, defns)\right]$

cbody⟦ ($id_{beg}\ stx\ ...$) @$ph\ body_{rest}\ ..., mname, deps, defns, M, \Sigma$⟧
= cbody⟦$stx@ph\ ...\ body_{rest}\ ..., mname, deps, defns, M, \Sigma$⟧
where $\langle ph,$ `base`, `begin`$\rangle$ = resolve⟦id_{beg}, ph⟧

cbody⟦ ($id_{bfs}\ stx\ ...$) @$ph\ body_{rest}\ ..., mname, deps, defns, M, \Sigma$⟧
= cbody⟦$stx@ph{+}1\ ...\ body_{rest}\ ..., mname, deps, defns, M, \Sigma$⟧
where $\langle ph,$ `base`, `begin-for-syntax`$\rangle$ = resolve⟦id_{bfs}, ph⟧

cbody⟦ ($id_{def}\ id_{new}\ stx$) @$ph\ body_{rest}\ ..., mname, deps, defn\ ..., M, \Sigma$⟧
= cbody⟦$body_{new}\ ..., mname, deps, defn\ ...\ defn_{new}, M, \Sigma_{new}$⟧
where $\langle ph,$ `base`, `define`$\rangle$ = resolve⟦id_{def}, ph⟧,
$dname$ = strip⟦id_{new}⟧, ast = cexpr⟦$stx, ph, \varepsilon, \Sigma$⟧,
$defn_{new} = \langle dname@ph, \textbf{VALUE}, ast\rangle$,
$binding_{new} = \langle ph, mname, dname\rangle$,
$body_{new}\ ...$ = add⟦$body_{rest}, binding_{new}$⟧ ...,
val = maybe-eval⟦ph, ast, Σ⟧,
$\Sigma_{new} = \Sigma\left[binding_{new} \leftarrow \langle \textbf{VALUE}, val\rangle\right]$

cbody⟦ (id_{defstx} id_{new} stx) @ph $body_{rest}$ ..., $mname$, $deps$, $defn$ ..., M, Σ⟧
= cbody⟦$body_{new}$ ..., $mname$, $deps$, $defn$ ... $defn_{new}$, M, Σ_{new}⟧
where ⟨ph, `base`, `define-syntax`⟩ = resolve⟦id_{defstx}, ph⟧,
$dname$ = strip⟦id_{new}⟧, ast = cexpr⟦stx, ph+1, ε, Σ⟧,
$defn_{new}$ = ⟨$dname$@ph, **MACRO**, ast⟩,
$binding_{new}$ = ⟨ph, $mname$, $dname$⟩,
$body_{new}$ ... = add⟦$body_{rest}$, $binding_{new}$⟧ ..., val = eval⟦ast, Σ⟧,
Σ_{new} = Σ[$binding_{new}$ ⟵ ⟨**MACRO**, val⟩]

cbody⟦ (id_{macro} stx ...) @ph $body_{rest}$ ..., $mname$, $deps$, $defns$, M, Σ⟧
= cbody⟦stx_{new}@ph $body_{rest}$ ..., $mname$, $deps$, $defns$, M, Σ⟧
where $binding$ = resolve⟦id_{macro}, ph⟧,
⟨**MACRO**, $func$⟩ = Σ($binding$),
SYNTAX(stx_{new}) = eval⟦**APP**($func$, **SYNTAX**((id_{macro} stx ...))), Σ⟧

cbody⟦ (id_{req} stx_{in}) @ph $body_{rest}$ ..., $mname$, dep ..., $defns$, M, Σ⟧
= cbody⟦$body_{new}$ ..., $mname$, dep ... dep_{new}, $defns$, M, Σ_{new}⟧
where ⟨ph, `base`, `require`⟩ = resolve⟦id_{req}, ph⟧,
⟨$mname_{in}$, ph_{Δ}⟩ = parse-req⟦stx_{in}, ph⟧,
Σ_{new} = visit⟦$mname_{in}$, ph_{Δ}, M, Σ⟧,
$bindings_{new}$ = import⟦$mname_{in}$, ph_{Δ}, M⟧,
$body_{new}$ ... = add⟦$body_{rest}$, $bindings_{new}$⟧ ...,
dep_{new} = $mname_{in}$@ph_{Δ}

cbody⟦ (id_{mod} id_{sub} id_{init} stx ...) @ph $body_{rest}$ ..., $mname$, $deps$, $defns$, M, Σ⟧
= cbody⟦$body_{rest}$ ..., $mname$, $deps$, $defns$, M_{new}, Σ⟧
where ⟨ph, `base`, `module`⟩ = resolve⟦id_{mod}, ph⟧,
$mname_{sub}$ = strip⟦id_{sub}⟧, $mname_{init}$ = strip⟦id_{init}⟧,
M_{new} = cmodule⟦ (`module` $mname_{sub}$ $mname_{init}$ strip⟦stx⟧ ...) , M⟧

cbody⟦ (id_{mod} id_{sub} id_{init} stx ...) @ph $body_{rest}$ ..., $mname$, $deps$, $defns$, M, Σ⟧
= cmodule⟦ (`module` $mname_{sub}$ $mname_{init}$ strip⟦stx⟧ ...) , M_{new}⟧
where ⟨ph, `base`, `module*`⟩ = resolve⟦id_{mod}, ph⟧,
$mname_{sub}$ = strip⟦id_{sub}⟧, $mname_{init}$ = strip⟦id_{init}⟧,
M_{new} = cbody⟦$body_{rest}$ ..., $mname$, $deps$, $defns$, M, Σ⟧

cbody⟦ (id_{mod} id_{sub} ( ) stx ...) @ph $body_{rest}$ ..., $mname$, $deps$, $defns$, M, Σ⟧
= cbody⟦shift⟦stx, 0–ph⟧@0 ..., $mname_{sub}$, $mname$@0–ph, ε, M_{new}, Σ_{init}⟧
where ⟨ph, `base`, `module*`⟩ = resolve⟦id_{mod}, ph⟧,
$mname_{sub}$ = strip⟦id_{sub}⟧,
M_{new} = cbody⟦$body_{rest}$ ..., $mname$, $deps$, $defns$, M, Σ⟧,
Σ_{init} = visit⟦$mname$, 0–ph, M_{new}, ∅⟧

---

cexpr : stx ph env Σ → ast

cexpr⟦id, ph, env, Σ⟧ = var
where var = env-lookup⟦env, **VAR**(strip⟦id⟧)⟧

$$\text{cexpr}[\![id, ph, env, \Sigma]\!] = \textbf{REF}(binding)$$
where $binding = \text{resolve}[\![id, ph]\!]$

$$\text{cexpr}[\![(\ id_{lambda}\ (\ id\ ...\ )\ stx\ )\ , ph, name_{env}\ ..., \Sigma]\!]$$
$$= \textbf{LAMBDA}(\textbf{VAR}(\text{strip}[\![id]\!])\ ..., \text{cexpr}[\![stx, ph, \textbf{VAR}(\text{strip}[\![id]\!])\ ...\ name_{env}\ ..., \Sigma]\!])$$
where $\langle ph, \texttt{base}, \texttt{lambda}\rangle = \text{resolve}[\![id_{lambda}, ph]\!]$

$$\text{cexpr}[\![(\ id_{quote}\ stx\ )\ , ph, env, \Sigma]\!] = \textbf{QUOTE}(\text{strip}[\![stx]\!])$$
where $\langle ph, \texttt{base}, \texttt{quote}\rangle = \text{resolve}[\![id_{quote}, ph]\!]$

$$\text{cexpr}[\![(\ id_{qstx}\ stx\ )\ , ph, env, \Sigma]\!] = \textbf{SYNTAX}(stx)$$
where $\langle ph, \texttt{base}, \texttt{syntax}\rangle = \text{resolve}[\![id_{qstx}, ph]\!]$

$$\text{cexpr}[\![(\ id_{macro}\ stx\ ...\ )\ , ph, env, \Sigma]\!] = \text{cexpr}[\![stx_{new}, ph, env, \Sigma]\!]$$
where $binding = \text{resolve}[\![id_{macro}, ph]\!]$,
$\langle \textbf{MACRO}, func\rangle = \Sigma(binding)$,
$\textbf{SYNTAX}(stx_{new}) = \text{eval}[\![\textbf{APP}(func, \textbf{SYNTAX}(\ (\ id_{macro}\ stx\ ...\ )\ ))\ , \Sigma]\!]$

$$\text{cexpr}[\![(\ stx\ ...\ )\ , ph, env, \Sigma]\!] = \textbf{APP}(\text{cexpr}[\![stx, ph, env, \Sigma]\!], ...)$$

$$\text{cexpr}[\![id, ph, env, \Sigma]\!] = \textbf{UNBOUND}$$

$$\text{eval} : ast\ \Sigma \rightarrow val$$

$$\text{eval}[\![val, \Sigma]\!] = val$$

$$\text{eval}[\![\textbf{REF}(binding), \Sigma]\!] = val$$
where $\langle \textbf{VALUE}, val\rangle = \Sigma(binding)$

$$\text{eval}[\![\textbf{APP}(ast, ast_{arg}, ...), \Sigma]\!] = \text{eval}[\![ast_{body}[\![var\ ... \leftarrow val\ ...]\!], \Sigma]\!]$$
where $\textbf{LAMBDA}(var\ ..., ast_{body}) = \text{eval}[\![ast, \Sigma]\!]$,
$val\ ... = \text{eval}[\![ast_{arg}, \Sigma]\!]\ ...$

$$\text{maybe-eval} : ph\ ast\ \Sigma \rightarrow val$$

$$\text{maybe-eval}[\![0, ast, \Sigma]\!] = \textbf{UNBOUND}$$

$$\text{maybe-eval}[\![ph, ast, \Sigma]\!] = \text{eval}[\![ast, \Sigma]\!]$$

$$\text{ast-shift} : ast\ ph \rightarrow ast$$

$$\text{ast-shift}[\![var, ph]\!] = var$$

$$\text{ast-shift}[\![\textbf{LAMBDA}(var\ ..., ast), ph]\!] = \textbf{LAMBDA}(var\ ..., \text{ast-shift}[\![ast, ph]\!])$$

$$\text{ast-shift}[\![\textbf{QUOTE}(name), ph]\!] = \textbf{QUOTE}(name)$$

ast-shift⟦**APP**$(ast, ...), ph$⟧ = **APP**(ast-shift⟦ast, ph⟧, ...)

ast-shift⟦**REF**$(\langle ph_{ref}, mname, dname\rangle), ph$⟧ = **REF**$(\langle ph_{ref}+ph, mname, dname\rangle)$

ast-shift⟦**SYNTAX**$(stx), ph$⟧ = **SYNTAX**(shift⟦stx, ph⟧)

ast-shift⟦**UNBOUND**$, ph$⟧ = **UNBOUND**

---

visit : $mname\ ph\ M\ \Sigma \rightarrow \Sigma$

visit⟦$mname, ph_{\Delta}, M, \Sigma$⟧ = Σ
 where **READY** = $\Sigma(mname@ph_{\Delta})$

visit⟦$mname, ph_{\Delta}, M, \Sigma$⟧ = visit-body⟦$defn\ ..., mname, ph_{\Delta}, \Sigma_{deps}$⟧
 where **MODULE**$(mname, deps, defn\ ...) = M(mname)$,
 $\Sigma_{init} = \Sigma\left[mname@ph_{\Delta} \leftarrow \textbf{READY}\right]$,
 Σ_{deps} = visit*⟦$deps, ph_{\Delta}, M, \Sigma_{init}$⟧

---

visit* : $deps\ ph\ M\ \Sigma \rightarrow \Sigma$

visit*⟦$\varepsilon, ph_{\Delta}, M, \Sigma$⟧ = Σ

visit*⟦$mname@ph\ dep\ ..., ph_{\Delta}, M, \Sigma$⟧ = visit*⟦$dep\ ..., ph_{\Delta}, M, \Sigma_{new}$⟧
 where Σ_{new} = visit⟦$mname, ph_{\Delta}+ph, M, \Sigma$⟧

---

visit-body : $defns\ name\ ph\ \Sigma \rightarrow \Sigma$

visit-body⟦$\varepsilon, mname, ph_{\Delta}, \Sigma$⟧ = Σ

visit-body⟦$\langle dname@ph_{def}$, **VALUE**$, ast\rangle\ defn\ ..., mname, ph_{\Delta}, \Sigma$⟧
 = visit-body⟦$defn\ ..., mname, ph_{\Delta}, \Sigma$⟧
 where $0 = ph_{\Delta}+ph_{def}$

visit-body⟦$\langle dname@ph_{def}, kind, ast\rangle\ defn\ ..., mname, ph_{\Delta}, \Sigma$⟧
 = visit-body⟦$defn\ ..., mname, ph_{\Delta}, \Sigma$⟧
 where -1 = pmax⟦$-1, ph_{\Delta}+ph_{def}$⟧

visit-body⟦$\langle dname@ph_{def}, kind, ast\rangle\ defn\ ..., mname, ph_{\Delta}, \Sigma$⟧
 = visit-body⟦$defn\ ..., mname, ph_{\Delta}, \Sigma_{new}$⟧
 where $ph = ph_{\Delta}+ph_{def}$, val = eval⟦shift⟦ast, ph_{Δ}⟧, Σ⟧,
 $\Sigma_{new} = \Sigma\left[\langle ph, mname, dname\rangle \leftarrow \langle kind, val\rangle\right]$

---

run : $mname\ ph\ M\ \Sigma \rightarrow \Sigma$

run⟦$mname, ph_{\Delta}, M, \Sigma$⟧ = Σ
 where **READY** = $\Sigma(mname@ph_{\Delta})$

$\mathsf{run}[\![mname, ph_\Delta, M, \Sigma]\!] = \mathsf{run\text{-}body}[\![defn\ ..., mname, ph_\Delta, \Sigma_{deps}]\!]$
where $\textbf{MODULE}(mname, deps, defn\ ...) = M(mname)$,
$\Sigma_{init} = \Sigma\big[mname@ph_\Delta \longleftarrow \textbf{READY}\big]$,
$\Sigma_{deps} = \mathsf{run}^*[\![deps, ph_\Delta, M, \Sigma_{init}]\!]$

$\mathsf{run}^* : deps\ ph\ M\ \Sigma \to \Sigma$

$\mathsf{run}^*[\![\varepsilon, ph_\Delta, M, \Sigma]\!] = \Sigma$

$\mathsf{run}^*[\![mname@ph\ dep\ ..., ph_\Delta, M, \Sigma]\!] = \mathsf{run}^*[\![dep\ ..., ph_\Delta, M, \Sigma_{new}]\!]$
where $\Sigma_{new} = \mathsf{run}[\![mname, ph_\Delta{+}ph, M, \Sigma]\!]$

$\mathsf{run\text{-}body} : defns\ name\ ph\ \Sigma \to \Sigma$

$\mathsf{run\text{-}body}[\![\varepsilon, mname, ph_\Delta, \Sigma]\!] = \Sigma$

$\mathsf{run\text{-}body}[\![\langle dname@ph_{def}, \textbf{VALUE}, ast\rangle\ defn\ ..., mname, ph_\Delta, \Sigma]\!]$
$= \mathsf{run\text{-}body}[\![defn\ ..., mname, ph_\Delta, \Sigma_{new}]\!]$
where $0 = ph_\Delta{+}ph_{def}$, $val = \mathsf{eval}[\![\mathsf{shift}[\![ast, ph_\Delta]\!], \Sigma]\!]$,
$\Sigma_{new} = \Sigma\big[\langle 0, mname, dname\rangle \longleftarrow \langle \textbf{VALUE}, val\rangle\big]$

$\mathsf{run\text{-}body}[\![defn_{skip}\ defn\ ..., mname, ph_\Delta, \Sigma]\!]$
$= \mathsf{run\text{-}body}[\![defn\ ..., mname, ph_\Delta, \Sigma]\!]$

$\mathsf{wrap} : s\text{-}exp\ bindings \to stx$

$\mathsf{wrap}[\![name, bindings]\!] = \textbf{STX}(name, bindings)$

$\mathsf{wrap}[\![(s\text{-}exp\ ...), bindings]\!] = (\mathsf{wrap}[\![s\text{-}exp, bindings]\!]\ ...)$

$\mathsf{strip} : stx \to s\text{-}exp$

$\mathsf{strip}[\![\textbf{STX}(name, bindings)]\!] = name$

$\mathsf{strip}[\![(stx\ ...)]\!] = (\mathsf{strip}[\![stx]\!]\ ...)$

$\mathsf{add\text{-}to\text{-}stx} : stx\ bindings \to stx$

$\mathsf{add\text{-}to\text{-}stx}[\![\textbf{STX}(name, binding\ ...), binding_{new}\ ...]\!]$
$= \textbf{STX}(name, binding\ ...\ binding_{new}\ ...)$

$\mathsf{add\text{-}to\text{-}stx}[\![(stx\ ...), bindings]\!] = (\mathsf{add\text{-}to\text{-}stx}[\![stx, bindings]\!]\ ...)$

$\mathsf{add} : body\ bindings \to body$

$\mathsf{add}[\![stx@ph, bindings]\!] = \mathsf{add\text{-}to\text{-}stx}[\![stx, bindings]\!]@ph$

---

resolve : $id\ ph \to maybe\text{-}binding$

resolve⟦$\mathbf{STX}(dname, binding_{pre}\ ...\ binding\ binding_{post}\ ...), ph$⟧ = $binding$
 where $\langle ph, mname, dname\rangle = binding$

resolve⟦id, ph⟧ = #f

---

shift : $stx\text{-}or\text{-}ast\ ph \to stx\text{-}or\text{-}ast$

shift⟦$\mathbf{STX}(name, \langle ph, mname, dname\rangle\ ...), ph_{shift}$⟧
 = $\mathbf{STX}(name, \langle ph{+}ph_{shift}, mname, dname\rangle\ ...)$

shift⟦($stx\ ...$), ph_{shift}⟧ = (shift⟦stx, ph_{shift}⟧ ...)

shift⟦ast, ph_{shift}⟧ = ast-shift⟦ast, ph_{shift}⟧

---

parse-req : $stx\ ph \to \langle name, ph\rangle$

parse-req⟦id_{in}, ph⟧ = ⟨strip⟦id_{in}⟧, ph⟩

parse-req⟦($id_{fs}\ stx_{in}$), ph⟧ = parse-req⟦$stx_{in}, ph{+}1$⟧
 where `for-syntax` = strip⟦id_{fs}⟧

parse-req⟦($id_{ft}\ stx_{in}$), ph⟧ = parse-req⟦$stx_{in}, ph{-}1$⟧
 where `for-template` = strip⟦id_{ft}⟧

---

import : $mname\ ph_{\Delta}\ M \to bindings$

import⟦$mname, ph_{\Delta}, M$⟧ = $\langle ph_{\Delta}{+}ph_{def}, mname, dname\rangle$ ...
 where $\mathbf{MODULE}(mname, dep\ ..., defn\ ...) = M(mname)$,
 $\langle dname@ph_{def}, kind, ast\rangle\ ... = defn\ ...$